计算机系统 I：算术运算单元 (Arithmetic Unit) 复习讲义

目录

1 运算单元总览与迭代组合电路 (Overview & Iterative Combinational Circuits) 2
1.1 本讲主线 (Lecture Roadmap) . . . 2
1.2 迭代组合电路 (Iterative Combinational Circuits) . . . 2

2 加法器 (Adders) 2
2.1 半加器 (Half-Adder) . . . 2
2.2 全加器 (Full Adder) . . . 3
2.3 多位进位传播加法器 (Carry Propagate Adders, CPA) . . . 3
2.4 行波进位加法器 (Ripple-Carry Adder, RCA) . . . 4
2.5 超前进位加法器 (Carry Lookahead Adder, CLA) . . . 4
2.6 组超前进位 (Group Carry Lookahead) . . . 5
2.7 前缀加法器 (Prefix Adder) . . . 5
2.8 Carry Skip 与 Carry Select . . . 6
2.9 32 位 CLA 与延迟比较 . . . 6

3 减法、溢出与标志位 (Subtraction, Overflow & Flags) 6
3.1 无符号补码减法 (Unsigned 2's Complement Subtraction) . . . 6
3.2 统一加减法硬件 (Adder-Subtractor) . . . 7
3.3 进位与溢出 (Carry & Overflow) . . . 7
3.4 带符号溢出检测 (Detecting Signed Overflow) . . . 7
3.5 重要标志位 (Important Flags) . . . 8
3.6 半减器与全减器 (Half/Full Subtractor) . . . 8

4 算术逻辑单元与移位器 (ALU & Shifters) 8
4.1 什么是 ALU? . . . 8
4.2 简单 1 位 ALU . . . 9
4.3 N 位 8 操作 ALU . . . 9
4.4 SLT 指令示例 (Set Less Than) . . . 9

4.5 移位器的用途 (Various Use of Shifters) 10
4.6 移位作为乘除法 (Shifters as Multipliers and Dividers) 10
4.7 组合移位器与桶形移位器 (Combinational and Barrel Shifters) 11
4.8 数据通路中的 ALU (Datapath and ALU) 11

5 乘法器 (Multipliers) 11
5.1 二进制乘法基础 11
5.2 直接竖式算法 (Straightforward Algorithm) 12
5.3 逐次乘法硬件实现 1 (Implementation 1) 12
5.4 逐次乘法硬件实现 2 与 3 13
5.5 有符号乘法与 Booth 算法 13
5.6 Booth 算法为什么成立? 15
5.7 Booth 示例与易错点 15
5.8 阵列乘法器与更快乘法 (Array Multiplier & Faster Multiplication) 15

6 除法器 (Dividers) 15
6.1 无符号除法与恢复除法 (Unsigned and Restoring Division) 15
6.2 恢复除法实现 1 16
6.3 恢复除法实现 2 与改进 16
6.4 非恢复除法 (Non-Restoring Division) 17
6.5 有符号除法 (Signed Division) 17
6.6 4 x 4 阵列除法器 17

7 浮点运算单元 (Floating-Point Operations) 18
7.1 浮点加法步骤 (Floating-Point Addition) 18
7.2 浮点加法硬件 19
7.3 浮点乘法步骤 (Floating-Point Multiplication) 19
7.4 浮点乘法、除法与硬件复杂度 21
7.5 浮点除法概要 (Floating-Point Division) 21

8 全章总结 (Summary) 21

1 运算单元总览与迭代组合电路 (Overview & Iterative Combinational Circuits)

1.1 本讲主线 (Lecture Roadmap)

本讲围绕处理器中负责计算的硬件展开，课件内容覆盖：

- **加法 (Addition)**：半加器、全加器、多位加法器、进位传播与快速进位。
- **减法 (Subtraction)**：无符号减法、补码减法、减法器与借位。
- **算术逻辑单元 (Arithmetic Logic Unit, ALU)**：将算术、逻辑、移位、比较等操作统一到一个可控数据通路中。
- **乘法 (Multiplication)**：移位加法、逐次乘法器、Booth 编码、阵列乘法器。
- **除法 (Division)**：恢复除法、非恢复除法、有符号除法。
- **浮点运算 (Floating-Point Operations)**：浮点加法、乘法、除法与硬件复杂度。

1.2 迭代组合电路 (Iterative Combinational Circuits)

算术函数通常作用在二进制向量上，并且每一位位置经常重复相同的子功能。因此可以先设计一个处理单个位或少数位的**单元 (Cell)**，再将多个单元互连形成完整的**迭代阵列 (Iterative Array)**。

- **Cell**：完成每一位的局部逻辑，如 1 位全加器。
- **Iterative Array**：由多个 cell 重复连接而成，可以是一维阵列，也可以是二维阵列。
- **典型例子**：行波进位加法器是一维阵列；阵列乘法器是二维阵列。

复习抓手

本讲的核心问题不是“如何手算加减乘除”，而是“硬件如何用有限的门、寄存器、移位器和控制信号完成这些运算”。复习时要同时关注：功能正确性、标志位含义、延迟、硬件面积和可流水化程度。

2 加法器 (Adders)

2.1 半加器 (Half-Adder)

半加器是 2 输入、1 位宽的二进制加法器，用于计算 $A + B$，输出一个和位 F 与一个进位位 C。

A	B	**和位** F	**进位** C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

逻辑表达式为：

$$F = A \oplus B, \qquad C = AB$$

2.2 全加器 (Full Adder)

全加器在半加器基础上加入来自低位的进位输入 C_{in}，输出和位 F 与进位输出 C_{out}。

A	B	C_{in}	F	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

常用表达式：

$$F = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

2.3 多位进位传播加法器 (Carry Propagate Adders, CPA)

多位加法器由多个 1 位加法单元连接构成。不同结构的核心差异在于如何处理进位。

类型	基本思想	特点
Ripple-Carry Adder (RCA)	将全加器串联，低位进位逐级传到高位。	硬件简单，但慢。
Carry Skip Adder	当一个块整体能传播进位时，让进位跳过该块内部。	比 RCA 快，块长可不均匀优化。
Carry Select Adder (CSA)	对每个块同时假设输入进位为 0 和 1，最后用选择器选结果。	速度快，硬件冗余。
Carry-Lookahead Adder (CLA)	用 generate/propagate 方程提前算出进位。	快，门扇入和硬件复杂度上升。
Prefix Adder	通过前缀网络并行计算每一列的进位。	很快，约 $\log_2 N$ 级。

2.4 行波进位加法器 (Ripple-Carry Adder, RCA)

RCA 将 N 个 1 位全加器串起来，每一级必须等待低位进位到达后才能确定结果。

- **优点**：结构最简单，面积小。
- **缺点**：最坏路径是进位穿过整个链，因此速度慢。
- **延迟公式**：

$$t_{ripple} = Nt_{FA}$$

其中 t_{FA} 是 1 位全加器的延迟。

2.5 超前进位加法器 (Carry Lookahead Adder, CLA)

CLA 的关键是将每一位的进位行为拆成**生成 (Generate)** 与**传递 (Propagate)**。

$$P_i = A_i \oplus B_i, \qquad G_i = A_i B_i$$

$$S_i = P_i \oplus C_i, \qquad C_{i+1} = G_i + P_i C_i$$

- 若 $A_i = B_i = 1$，则无论 C_i 是多少，高位进位都会被生成，即 $G_i = 1$。
- 若 $A_i \oplus B_i = 1$ 且 $C_i = 1$，则输入进位会被传播到高位，即 $P_i C_i = 1$。

从 C_0 开始展开：

$$\begin{aligned}
C_1 &= G_0 + P_0 C_0 \\
C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\
C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\
C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0
\end{aligned}$$

2.6 组超前进位 (Group Carry Lookahead)

直接展开到很多位会遇到门扇入限制，因此课件引入组生成与组传递。对 4 位块 $0 \sim 3$：

$$G_{0\sim3} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

$$P_{0\sim3} = P_3P_2P_1P_0$$

于是：

$$C_4 = G_{0\sim3} + P_{0\sim3}C_0$$

16 位 CLA 可由四个 4 位 CLA 块组成：

$$C_4 = G_{0\sim3} + P_{0\sim3}C_0$$
$$C_8 = G_{4\sim7} + P_{4\sim7}C_4$$
$$C_{12} = G_{8\sim11} + P_{8\sim11}C_8$$
$$C_{16} = G_{12\sim15} + P_{12\sim15}C_{12}$$

课件给出的 16 位 CLA 延迟例子中，假设 `NOT`=1，`XOR`=3，孤立 `AND`=3，`AND-OR`=2：

- RCA 最长延迟：$3 + 15 \times 2 + 3 = 36$。
- CLA 最长延迟：$3 + 3 \times 2 + 3 = 12$。

2.7 前缀加法器 (Prefix Adder)

前缀加法器先计算每一列的进位，再计算和位：

$$S_i = (A_i \oplus B_i) \oplus C_{i-1}$$

课件将第 -1 列定义为输入进位：

$$G_{-1} = C_{in}, \qquad P_{-1} = 0$$

列 i 的输入进位等于列 $i-1$ 到 -1 的组生成：

$$C_{i-1} = G_{i-1:-1}$$

对跨越 $i:j$ 的块，若以 k 为分界：

$$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j}$$

$$P_{i:j} = P_{i:k}P_{k-1:j}$$

前缀加法器延迟近似为：

$$t_{PA} = t_{pg} + \log_2 N \cdot t_{pg_prefix} + t_{XOR}$$

2.8 Carry Skip 与 Carry Select

- **Carry Skip Adder**：若某一块所有位都能传播进位，则进位无需在块内部逐级传播，可直接跳到下一块。为了最优速度，块长通常不必相等。
- **Carry Select Adder**：将每一块在 $C_{in}=0$ 和 $C_{in}=1$ 两种情况下都预先算好，再根据真实进位选择结果。课件中特别提到可以按 nibble（4 位块）做选择。

2.9 32 位 CLA 与延迟比较

补充页中给出 32 位 CLA 可由多个 4 位块构成。其延迟模型为：

$$t_{CLA}=t_{pg}+t_{pg_block}+\left(\frac{N}{k}-1\right)t_{AND_OR}+kt_{FA}$$

- t_{pg}：生成所有 P_i, G_i 的延迟。
- t_{pg_block}：生成块级 $P_{i:j}, G_{i:j}$ 的延迟。
- t_{AND_OR}：最终 AND/OR 门中从 C_{in} 到 C_{out} 的延迟。
- k：每个 CLA 块的位数。

课件总结：若只按门级数估计，RCA 与 CLA 的 C_i、S_i 门级可能分别是 RCA 的 $(2,1)$ 与 CLA 的 $(3,4)$；但真实延迟还取决于门类型与负载。一般当 $N>16$ 时，CLA 明显快于 RCA。

3 减法、溢出与标志位 (Subtraction, Overflow & Flags)

3.1 无符号补码减法 (Unsigned 2's Complement Subtraction)

对 n 位无符号数 $M-N$，可将减法转化为加法：

$$M-N=M+(2^n-N)-2^n$$

也就是给被减数 M 加上减数 N 的二进制补码。

- 若 $M>N$，加法产生最高位进位 $r_n=1$，丢弃该进位后得到正结果 $M-N$。
- 若 $M<N$，没有最高位进位，得到的是 $N-M$ 的补码；若要写成普通带负号形式，需要对结果再取补码并在前面加负号。

例 1： $01010100_2-01000011_2$

$$01010100+10111101=1\,00010001$$

最高位进位为 1，丢弃进位，结果为：

$$00010001_2=17_{10}$$

例 2： $01000011_2 - 01010100_2$

$$01000011 + 10101100 = 0\,11101111$$

最高位进位为 0，需要修正。对 11101111 取补码得 00010001，因此结果为：

$$-00010001_2 = -17_{10}$$

3.2 统一加减法硬件 (Adder-Subtractor)

补码表示使加法和减法可以使用同一套加法器：

- 当控制位 $S = 0$：执行 $A + B$。
- 当控制位 $S = 1$：将 B 各位取反，并令最低位进位 $C_0 = 1$，执行

$$A + \bar{B} + 1 = A - B$$

- 该电路既可用于无符号数加减，也可用于带符号补码数加减；区别在于如何解释结果与标志位。

3.3 进位与溢出 (Carry & Overflow)

- **Carry** 主要用于无符号整数，表示无符号结果是否超出 n 位范围。
- **Overflow** 主要用于带符号补码整数，表示带符号结果是否超出可表示范围。

以 4 位补码为例，范围为 $[-8, +7]$。计算 $4 + 5$ 得到位模式 1001，若按补码解释为 -7，发生正溢出；计算 $(-4) + (-5)$ 得到位模式 0111，若按补码解释为 $+7$，发生负溢出。

3.4 带符号溢出检测 (Detecting Signed Overflow)

补码加法中，溢出只会发生在以下两种情况：

- 两个正数相加，结果为负。
- 两个负数相加，结果为正。

等价判定：

- 若两个操作数符号位相同，而结果符号位不同，则发生溢出。
- 或者比较符号位的进位输入与进位输出：

$$OF = V = C_n \oplus C_{n-1}$$

Carry 和 Overflow 不可混用

无符号加法看 **Carry Flag**，带符号补码加法看 **Overflow Flag**。课件中特别强调：带符号运算不能只用最高位进位 C_n 判断溢出。

3.5 重要标志位 (Important Flags)

标志位	含义与适用场景
ZF (Zero Flag)	当结果为 0 时置 1；无符号与带符号运算都适用。
SF/NF (Sign Flag)	结果的最高位 S_{n-1}；主要用于带符号运算。
CF (Carry/Borrow Flag)	加法中通常表示 C_{out}；减法中常表示借位，课件写作 $\sim C_{out}$；主要用于无符号运算。
OF (Overflow Flag)	表示带符号补码运算溢出；主要用于带符号运算。

3.6 半减器与全减器 (Half/Full Subtractor)

补充页给出直接构造减法器的方法。半减器计算 $X - Y$：

X	Y	差 D	借位 B_{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

半减器公式：

$$D = X \oplus Y, \qquad B_{out} = \bar{X}Y$$

全减器计算 $X - Y - B_{in}$：

$$D = X \oplus Y \oplus B_{in}$$

$$B_{out} = \bar{X}Y + \bar{X}B_{in} + YB_{in}$$

n 位减法器可以用两种方式实现：

1. 串接 n 个全减器，形成 borrow ripple subtractor。
2. 使用 n 位加法器和 n 个反相器，将 Y 取反并令最低位进位为 1，从而计算 $X + \bar{Y} + 1$。

4 算术逻辑单元与移位器 (ALU & Shifters)

4.1 什么是 ALU?

ALU (Arithmetic-Logic Unit) 执行多种算术与逻辑操作，是处理器的核心计算部件。课件中称它为处理器的 “heart”：CPU 的其他部分很大程度上是在为 ALU 提供操作数、控制信号和结果存储。

构造 ALU 的两类思路：

- **扩展加法器 (extend the adder)**：在加法器周围增加反相、逻辑门、选择器和标志位逻辑。
- **并行冗余选择 (parallel redundant select)**：多个功能并行计算，最终由多路选择器选择输出。

4.2 简单 1 位 ALU

课件给出一个支持 3 个操作的简单 ALU：

功能码 $F_{1:0}$	**功能**
00	$A\&B$
01	$A\|B$
10	$A+B$
11	未使用

在多位版本中，可同时输出 **OF**、**SF**、**ZF**、**CF** 等标志位。

4.3 N 位 8 操作 ALU

课件给出的 N 位 ALU 使用 3 位控制线选择 8 类操作：

ALU Control Lines	**Function**
000	And
001	Or
010	Add
110	Sub
111	SLT (Set Less Than)
100	not
101	srl (shift right logical)
011	xor

硬件结构上，逻辑门、加法/减法器、移位器、SLT 结果等输出会进入 MUX，由控制线选择最终结果，同时生成 Zero 与 Overflow 等信号。

4.4 SLT 指令示例 (Set Less Than)

课件示例：配置 32 位 ALU 执行 SLT，$A = 25$，$B = 32$。

- 因为 $A < B$，所以输出应为 32 位的 1，即 `0x00000001`。
- 对应 MIPS 风格语义：`slt rd, rs, rt`，若 $rs < rt$ 则 $rd = 1$，否则 $rd = 0$。

- ALU 内部可通过执行 $A - B$ 判断：$25 - 32 = -7$，结果符号位为 1，说明 $A < B$。
- 输出寄存器只有最低位为 1，其余位为 0。

补充注意：对带符号 SLT，若减法发生溢出，不能只看符号位，稳妥的比较位可写为：

$$set = S_{31} \oplus OF$$

4.5 移位器的用途 (Various Use of Shifters)

课件区分三类移位/旋转操作：

- **逻辑移位 (Logical Shifter)**：左移或右移后空位补 0。
  - `11001 >> 2 = 00110`
  - `11001 << 2 = 00100`
- **算术移位 (Arithmetic Shifter)**：左移与逻辑移位相同；右移时空位补旧最高有效位 MSB，用于保持补码符号。
  - `11001 >>> 2 = 11110`
  - `11001 <<< 2 = 00100`
- **循环移位 (Rotator)**：从一端移出的位从另一端补入。
  - `11001 ROR 2 = 01110`
  - `11001 ROL 2 = 00111`

4.6 移位作为乘除法 (Shifters as Multipliers and Dividers)

对二进制整数：

$$A << N = A \times 2^N$$

$$A >>> N \approx A \div 2^N$$

课件示例：

- `00001 << 2 = 00100`，即 $1 \times 2^2 = 4$。
- `11101 << 2 = 10100`，即 $-3 \times 2^2 = -12$。
- `01000 >>> 2 = 00010`，即 $8 \div 2^2 = 2$。
- `10000 >>> 2 = 11100`，即 $-16 \div 2^2 = -4$。

4.7 组合移位器与桶形移位器 (Combinational and Barrel Shifters)

传统双向移位寄存器带并行装载时，一个寄存器传输和移位操作可能需要多个时钟：

1. 从总线并行装载到移位寄存器。
2. 执行左移、右移或保持。
3. 将结果传送到目标寄存器。

若将移位器实现为组合电路，则信号直接通过门和 MUX 传播，不需要额外移位时钟，数据通路中的一次寄存器传输可更快完成。

4 位循环左移 (ROL) 桶形移位器的功能表：

S_1	S_0	$Y_3Y_2Y_1Y_0$	**操作**
0	0	$D_3D_2D_1D_0$	不旋转
0	1	$D_2D_1D_0D_3$	左旋 1 位
1	0	$D_1D_0D_3D_2$	左旋 2 位
1	1	$D_0D_3D_2D_1$	左旋 3 位

4.8 数据通路中的 ALU (Datapath and ALU)

CPU 可以分成两部分：

- **数据通路 (Datapath)**：寄存器、共享 ALU、移位器和互连总线。
- **控制单元 (Control Unit)**：产生选择信号，控制数据在总线、ALU、移位器和寄存器之间流动。

当 ALU 是组合电路时，从源寄存器经 ALU 到目标寄存器的寄存器传送操作可在一个时钟周期内完成。移位操作可放在独立单元中，也可整合到 ALU 内。

5 乘法器 (Multipliers)

5.1 二进制乘法基础

乘法比加法复杂，因为直接实现通常包含多次移位和加法。复杂运算会带来两类代价：

- 更大的硅片面积 (area)。
- 更长的运行时间，例如多周期完成，或拉长单个时钟周期。

二进制乘法与十进制竖式乘法类似：

- 用乘数的每一位与被乘数相乘，形成部分积。

- 将部分积按位权左移后求和，得到最终乘积。

若

$$A = \sum_{i=0}^{m-1} a_i 2^i, \qquad B = \sum_{j=0}^{n-1} b_j 2^j$$

则

$$A \cdot B = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} a_i b_j 2^{i+j}$$

无符号 m 位乘 n 位的结果最多需要 $m+n$ 位。

5.2 直接竖式算法 (Straightforward Algorithm)

课件例子：

`01010010`	multiplicand
$\times$ `01101101`	multiplier
`0100010111010 10`	full sum

含义是：从乘数最低位开始扫描，若当前位为 1，则把被乘数按当前位置左移后加入部分和；若当前位为 0，则加入 0。课件后面的逐页动画展示了 partial sum 如何随着每个乘数位逐步增加。

5.3 逐次乘法硬件实现 1 (Implementation 1)

第一种实现保存三个寄存器：

- **Multiplicand**：被乘数寄存器，逐轮左移。
- **Multiplier**：乘数寄存器，逐轮右移。
- **Product**：部分积寄存器。

每一轮：

1. 测试乘数最低位 `Multiplier0`。
2. 若为 1，将 multiplicand 加到 product。
3. multiplicand 左移 1 位。
4. multiplier 右移 1 位。
5. 重复到指定位数结束。

例：$0010_2 \times 0110_2 = 2 \times 6 = 12$，最终乘积为 `00001100`。
缺点：

- ALU 宽度是必要宽度的两倍。
- multiplicand 寄存器也需要两倍位宽。
- product 寄存器在最后一步前并不需要完整 $2n$ 位。
- multiplier 寄存器在过程中逐渐被清空。

5.4 逐次乘法硬件实现 2 与 3

Implementation 2 的思想是：

- 被乘数固定 (Multiplicand stationary)。
- 乘数右移 (Multiplier right)。
- 部分积右移 (Partial Product right)。
- 只需将 multiplicand 加到 product 的左半部分。

Implementation 3 进一步把乘数放在部分积寄存器的右半边：

- 初始 `Product|Multiplier` 为 “高半部分 0 + 低半部分乘数”。
- 每轮测试最低位，也就是当前 `Product0`。
- 若为 1，将 multiplicand 加到 product 左半部分。
- 整个 `Product|Multiplier` 右移 1 位。

对 $0010_2 \times 0110_2$，Implementation 3 最终也得到 `00001100`，但寄存器利用率更高。

5.5 有符号乘法与 Booth 算法

基本有符号乘法可以这样做：

1. 保存两个操作数的符号。
2. 将它们转成无符号绝对值。
3. 执行无符号乘法。
4. 若两个符号相同，结果为正；若符号不同，结果为负。

更好的方法是 **Booth's Algorithm**，前提是硬件支持加法和减法。它的思想是：当乘数中出现连续的 1 时，不必对每个 1 都加一次，而是在连续 1 的开始做一次减法，在连续 1 的结束做一次加法，中间只移位。

当前位	右侧上一位	含义	操作
1	0	一串 1 的开始	Subtract
1	1	一串 1 的中间	Nothing
0	1	一串 1 的结束	Add
0	0	一串 0 的中间	Nothing

1 位 Booth 编码：

Y_j	Y_{j-1}	**1-bit Encoding**
0	0	$+0$
0	1	$+1$
1	0	-1
1	1	-0

2 位 Booth 编码：

Y_{j+1}	Y_j	Y_{j-1}	**2-bit Encoding**
0	0	0	$+0$
0	0	1	$+1$
0	1	0	$+1$
0	1	1	$+2$
1	0	0	-2
1	0	1	-1
1	1	0	-1
1	1	1	-0

2 bits/cycle Booth 乘法器每轮处理两个乘数位：

- 编码为 -2：被乘数左移 1 位后相减。
- 编码为 -1：相减。
- 编码为 0：不操作。
- 编码为 $+1$：相加。
- 编码为 $+2$：被乘数左移 1 位后相加。

5.6 Booth 算法为什么成立?

若乘数为 $a_{31}a_{30}\ldots a_0$，普通表示为:

$$b(a_{31}a_{30}\ldots a_0) = a_0 b2^0 + a_1 b2^1 + \cdots + a_{31} b2^{31}$$

Booth 重写为相邻位差:

$$= (0 - a_0)b2^0 + (a_0 - a_1)b2^1 + \cdots + (a_{30} - a_{31})b2^{31} + a_{31}b2^{32}$$

当乘数中有长串连续的 1 时，这种表示会把很多个加法压缩成 “开头一次减法 + 结尾一次加法”，从而减少部分积数量。对负乘数也成立，因为补码的最高位本来就带有负权重。

5.7 Booth 示例与易错点

课件示例:

- $0010_2 \times 1101_2 = 2 \times (-3)$，Booth 逐轮得到结果 `11111010`，即 -6。
- 负被乘数例：$1010_2 \times 0110_2 = (-6) \times 6 = -36$，结果 `11011100`。
- 负乘数例：$1010_2 \times 1110_2 = (-6) \times (-2) = 12$，结果 `00001100`。

课件还提醒有符号部分积必须做**符号扩展**；如果乘数最高位为 1，最高位对应的部分积需要做减法而不是普通加法。

5.8 阵列乘法器与更快乘法 (Array Multiplier & Faster Multiplication)

阵列乘法器用基本 cell 同时产生并累加部分积。每个 cell 可看作由 A_iB_j、输入和 S_{in}、输入进位 C_{in} 进入全加器，输出 S_{out} 与 C_{out}。

- **优点**：概念直观，所有部分积并行推进。
- **缺点**：硬件成本较高。
- **加速方式**：使用多级加法器树，或流水线化，使多个乘法可并行处在不同阶段。
- **工程注意**：整数乘法相对昂贵，在数组地址计算中常可用移位替代乘以 2^k。

6 除法器 (Dividers)

6.1 无符号除法与恢复除法 (Unsigned and Restoring Division)

二进制除法类似竖式除法。硬件需要判断 “当前余数是否足够减去除数”:

$$\text{condition: remainder} \geq \text{divisor}$$

硬件通常通过减法判断：

$$\text{remainder} - \text{divisor} \geq 0$$

若减得动，则保留新余数并写入商位 1；若减不动，则必须把余数恢复到减法前的值，并写入商位 0。这就是**恢复除法 (Restoring Division)**。

6.2 恢复除法实现 1

Implementation 1 的流程：

1. 将 dividend 放入 remainder 寄存器低半部分。
2. 用 divisor 寄存器与 remainder 寄存器做减法。
3. 若 remainder ≥ 0，商寄存器左移并写入 1。
4. 若 remainder < 0，加回 divisor 恢复余数，商寄存器左移并写入 0。
5. divisor 右移 1 位。
6. 对 32 位除法重复到结束。

课件例子 $7 \div 2$，最终：

$$Q = 3, \qquad R = 1$$

6.3 恢复除法实现 2 与改进

Implementation 2 的改进：

- 跳过第一次无意义的 subtract，因为一开始不可能向商中移入 1。
- 先将 remainder 左移，再做减法。
- quotient 可存放在 remainder 寄存器低半部分。
- 只需 32 位 ALU。
- 结束时将 remainder 左半部分右移 1 位，撤销多余移位。

这种结构与逐次乘法器更相似，但恢复除法仍可能每一位需要两次 ALU 操作：一次试减，一次在失败时恢复。

6.4 非恢复除法 (Non-Restoring Division)

恢复除法的问题是失败时需要“加回去”。非恢复除法避免了这一步。

若当前余数试减后为负：

$$R_i = 2R_{i-1} - d < 0$$

下一步若不恢复，而是改做加法，可得到同样效果：

$$R_{i+1} = 2R_i + d$$

直观理解：如果上一步多减了一个 d，下一步把余数左移后会放大这个误差，因此通过“加 d”修正，而不是先恢复再减。

- 余数非负：下一步做减法，商位写 1。
- 余数为负：下一步做加法，商位写 0。
- 吞吐率可达到每周期产生 1 个商位。

6.5 有符号除法 (Signed Division)

最简单思路是先记录 dividend 与 divisor 的符号，对绝对值做无符号除法，再修正符号：

- 若两个操作数符号相反，则商为负。
- 若两个操作数符号相同，则商为正。
- 非零余数的符号应与 dividend 相同。

按“向 0 截断”的整数除法习惯：

- $(-7) \div 2 = -3$，余数 -1。
- $7 \div (-2) = -3$，余数 1。
- $(-7) \div (-2) = 3$，余数 -1。

6.6 4 x 4 阵列除法器

补充页给出 4 位除法器的抽象算法。对 A/B：

$$A/B = Q + R/B$$

算法：

1. $R' = 0$。

2. 对 $i = N-1$ 到 0：

$$R = \{R' << 1,\ A_i\}$$

$$D = R - B$$

3. 若 $D < 0$，则 $Q_i = 0,\ R' = R$。

4. 否则 $Q_i = 1,\ R' = D$。

5. 结束后得到商 Q 与余数 R。

7 浮点运算单元 (Floating-Point Operations)

7.1 浮点加法步骤 (Floating-Point Addition)

浮点加法的核心是：**先对阶，再加尾数，再规格化，再舍入**。

1. **Alignment**：比较指数，将指数较小的数的尾数右移，直到两个指数相同。

2. **Add Significands**：对齐后加/减尾数。

3. **Normalize**：若结果不符合规格化形式，移动尾数并调整指数。

4. **Over/Underflow Check**：检查指数是否溢出或下溢。

5. **Rounding**：按浮点舍入规则保留有限位数。

6. **Renormalization**：若舍入导致进位，可能需要再次规格化。

十进制示例

系统精度为 4 位，计算：

$$9.999 \times 10^1 + 1.610 \times 10^{-1}$$

对阶：

$$1.610 \times 10^{-1} = 0.01610 \times 10^1 \to 0.016 \times 10^1$$

加尾数：

$$9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$$

规格化：

$$10.015 \times 10^1 = 1.0015 \times 10^2$$

舍入：

$$1.0015 \times 10^2 \to 1.002 \times 10^2$$

二进制示例： $0.5 + (-0.4375)$

$$0.5_{10} = 1.000_2 \times 2^{-1}$$

$$-0.4375_{10} = -1.110_2 \times 2^{-2}$$

对阶：

$$-1.110_2 \times 2^{-2} \rightarrow -0.111_2 \times 2^{-1}$$

加尾数：

$$1.000_2 \times 2^{-1} + (-0.111_2 \times 2^{-1}) = 0.001_2 \times 2^{-1}$$

规格化：

$$0.001_2 \times 2^{-1} = 1.000_2 \times 2^{-4}$$

结果：

$$1.000_2 \times 2^{-4} = 0.0625_{10}$$

7.2 浮点加法硬件

FP Adder 的硬件步骤与算法一致：

- 比较指数，计算指数差。
- 右移较小数的尾数。
- 用大 ALU 对尾数做加/减。
- 规格化，必要时调整指数。
- 进入 rounding hardware。

课件强调：浮点加法器远比整数加法器复杂。若强行一个时钟周期完成，会显著拉长时钟周期，拖慢所有指令。因此 FP adder 通常多周期完成，并且常采用流水线。

7.3 浮点乘法步骤 (Floating-Point Multiplication)

浮点数可拆成符号、尾数、指数分别处理：

$$(s_1 \cdot 2^{e_1})(s_2 \cdot 2^{e_2}) = (s_1 s_2) \cdot 2^{e_1+e_2}$$

浮点乘法流程：

1. **Sign**：结果符号为两个操作数符号异或。
2. **Add Exponents**：指数相加；若使用偏置指数，需减去一次 bias。
3. **Multiply Significands**：尾数相乘。

4. **Normalize**：规格化乘积。
5. **Over/Underflow Check**：检查指数范围。
6. **Rounding**：舍入并在必要时再次规格化。

十进制示例

计算：

$$1.110 \times 10^{10} \times 9.200 \times 10^{-5}$$

指数相加：

$$10 + (-5) = 5$$

尾数相乘：

$$1.110 \times 9.200 = 10.212$$

得到：

$$10.212 \times 10^{5} = 1.0212 \times 10^{6}$$

舍入后：

$$1.021 \times 10^{6}$$

偏置指数示例

课件例子：

$$1\ 10000010\ 000\ldots000 = -1 \times 2^{3}$$

$$0\ 10000011\ 000\ldots000 = +1 \times 2^{4}$$

符号：$1 \oplus 0 = 1$，结果为负。尾数均为 1，乘积仍为 1。单精度 bias 为 127：

$$10000010_2 + 10000011_2 - 01111111_2 = 10000110_2$$

即 $130 + 131 - 127 = 134 = 127 + 7$。结果：

$$1\ 10000110\ 000\ldots000 = -1 \times 2^{7}$$

二进制乘法示例

计算 $0.5_{10} \times (-0.4375_{10})$：

$$0.5_{10} = 1.000_2 \times 2^{-1}$$

$$-0.4375_{10} = -1.110_2 \times 2^{-2}$$

指数：

$$-1 + (-2) = -3$$

尾数：

$$1.000_2 \times 1.110_2 = 1.110_2$$

符号为负，因此：

$$-1.110_2 \times 2^{-3} = -0.21875_{10}$$

7.4 浮点乘法、除法与硬件复杂度

FP multiplier 与 FP adder 复杂度相近，只是用尾数乘法器替代尾数加法器。浮点运算硬件通常支持：

- 加法、减法、乘法、除法。
- 倒数 (reciprocal)、平方根 (square-root)。
- 浮点数与整数之间的转换。

这些操作通常需要多个周期完成，也适合流水线化。

7.5 浮点除法概要 (Floating-Point Division)

浮点除法与乘法类似，只是指数和尾数处理方向相反：

1. **Subtract Exponents**：指数相减；若使用 bias，需要做偏置修正。
2. **Divide Significands**：尾数相除。
3. **Normalize**：规格化结果。
4. **Rounding**：舍入。
5. **Sign**：结果符号由两个操作数符号异或决定。

8 全章总结 (Summary)

本讲的主线可以压缩为“用同一套硬件思想构造多种算术操作”：

1. **加法器是基础**：半加器、全加器构成多位加法器；RCA 简单但慢，CLA/Prefix 通过提前计算进位提速。
2. **补码统一加减法**：减法可转成 $A + \bar{B} + 1$，同一套加法器既服务加法又服务减法。
3. **标志位必须按语义解释**：无符号看 CF，带符号看 OF，ZF 和 SF 则用于结果状态判断。
4. **ALU 是可控的组合计算核心**：逻辑、算术、比较、移位等功能并行或复用实现，最后由 MUX 选择输出。

5. **乘法是移位加法的系统化**：逐次乘法节省硬件但多周期；Booth 编码减少连续 1 的部分积；阵列乘法器用面积换速度。

6. **除法依赖试减与余数修正**：恢复除法失败时加回除数，非恢复除法用下一轮加法避免恢复步骤。

7. **浮点运算复杂得多**：对阶、规格化、舍入、溢出/下溢检查使 FP 单元通常多周期并流水化。

"算术单元的设计，本质是在正确性、速度、面积和控制复杂度之间做权衡。"